GUIA DO ALUNO

ESP32-CAM: da câmera de segurança ao reconhecimento de gestos

OV3660 · vídeo ao vivo · gravação no microSD · Google Drive e Telegram · Python com OpenCV e MediaPipe · gestos que acendem LEDs

---

Placa: ESP32-CAM AI-Thinker com sensor OV3660 (ou OV2640) • **PC:** Python 3 + OpenCV + MediaPipe

Projetos: câmera de segurança local • câmera na nuvem (Drive/Telegram) • gestos → LED RGB

Pré-requisitos: aulas de LEDs, sensores e página web no ESP32

Carga horária: 5 aulas de 100 min

Espaço Cria — 2026

Sumário

Como este curso funciona

Segurança e cuidados

Aula 1 — ESP32-CAM e OV3660: foto e vídeo ao vivo

Aula 2 — Câmera de segurança: PIR, microSD e galeria

Aula 3 — Câmera na internet: Google Drive e Telegram

Aula 4 — Python: o que um computador vê numa imagem

Aula 5 — Projeto: gestos com a mão acendem LEDs

Para ir além

Glossário

Anexo — Checklist

Como este curso funciona

O **ESP32-CAM** é um ESP32 com uma câmera, um slot microSD e um LED de flash, tudo por ~R$ 50. Com ele vamos construir, em cinco aulas, o caminho completo de uma **câmera inteligente**: capturar e transmitir imagem (Aula 1), reagir ao mundo e guardar evidências (Aula 2), enviar para a nuvem e para o seu celular (Aula 3), entender o que um computador "vê" numa imagem (Aula 4) e fechar o ciclo com **visão computacional** controlando hardware: gestos com a mão acendem LEDs de cores diferentes (Aula 5).

Aula	Tema	Você vai construir	Arquivos
1	ESP32-CAM e OV3660	Foto e vídeo ao vivo na rede local, controle de flash e resolução	01_cam_foto_stream
2	Câmera de segurança	PIR dispara foto → microSD com data/hora → galeria web; timelapse	02_cam_seguranca_sd
3	Câmera na internet	Foto no Google Drive (Apps Script) e alerta com foto no Telegram	03_cam_drive_telegram + Code.gs
4	Python vê imagens	Pixels, cinza, bordas, cor, rostos; stream da câmera no PC	reconhecer_imagem.py, ver_stream.py
5	Gestos → LEDs	MediaPipe reconhece a mão; cada gesto acende uma cor no ESP32-CAM	04_cam_gestos_leds + gestos_leds.py

Como usar este guia

Cada aula: **conceito → montagem (figura) → código → entenda → experimente → desafios** (com alertas por LED/buzzer e lógica). Os sketches do ESP32-CAM precisam do arquivo `camera_aithinker.h` na mesma pasta.

Placa na IDE: **AI Thinker ESP32-CAM** (esp32 by Espressif). Códigos em `aula-esp32cam/codigo/` e `aula-esp32cam/python/`.

Segurança e cuidados

- O ESP32-CAM precisa de **5 V e pelo menos 500 mA** (Wi-Fi + câmera + flash). Fonte fraca = *brownout* e reinícios. Use a saída 5 V de um adaptador FTDI bom ou uma fonte externa; nunca alimente pelo 3V3.
- O **cabo flat da câmera** é frágil: para abrir/fechar a trava, puxe a aba preta com cuidado; contatos dourados voltados para a placa.
- O LED de flash (GPIO 4) é forte: não olhe de perto; ele esquenta se ficar ligado.
- **Privacidade:** câmera aponta só para o que o grupo combinou; não filme colegas sem consentimento; fotos no Drive/Telegram ficam numa conta do laboratório e são apagadas ao fim do curso.
- Credenciais (Wi-Fi, token do Telegram, URL do Apps Script) são do laboratório — não publique.

Aula 1 — ESP32-CAM e OV3660: foto e vídeo ao vivo

O que você vai aprender

- A anatomia do ESP32-CAM: ESP32 + PSRAM + câmera + microSD + flash, e por que ele não tem USB.
- Como uma câmera digital forma a imagem (sensor CMOS, pixels, JPEG) e o que muda entre OV2640 e OV3660.
- Gravar o firmware com um adaptador USB-serial e o pulo do gato do **GPIO 0**.
- Servir foto (`/capture`) e vídeo (**MJPEG** em `/stream`) numa página web.

Hardware: o que há na placa

Bloco	O que é	Detalhe importante
ESP32-S	Dual-core 240 MHz, Wi-Fi/Bluetooth	Igual ao DevKit, mas quase todos os pinos vão para a câmera
PSRAM 4 MB	Memória externa	Onde cabe a imagem grande (um JPEG UXGA tem ~100–200 kB; o ESP32 só tem 520 kB de RAM)
Câmera OV3660	Sensor CMOS 3 MP (2048×1536), lente fixa, ~66°	Substitui o OV2640 (2 MP) em muitos kits; mesma interface (DVP + SCCB/I²C)
microSD	Slot 1-bit ou 4-bit	Em 4-bit usa GPIO 2, 4, 12, 13, 14, 15; em **1-bit** libera 4, 12 e 13 para você
Flash	LED branco no GPIO 4	Compartilhado com o SD (modo 4-bit)
Pinos livres	GPIO 12, 13, 14, 15 (sem SD), 2, 16, 33 (LED vermelho, ativo em LOW)	GPIO 12 e 0 são *strapping*: não devem estar em nível alto no boot
Serial	U0R (GPIO 3) / U0T (GPIO 1)	Gravação e Monitor Serial via adaptador externo

A física: como um sensor CMOS forma a imagem

Cada **pixel** do sensor é um fotodiodo: a luz arranca elétrons do silício (efeito fotoelétrico) e a carga acumulada durante o **tempo de exposição** vira uma tensão, que um ADC converte em número (8–10 bits). Sobre os pixels há um mosaico de filtros vermelho/verde/azul (**filtro de Bayer**: 50 % verde, porque o olho é mais sensível a ele); um processador no próprio sensor interpola as cores, ajusta ganho, balanço de branco e **comprime em JPEG** — por isso o ESP32 recebe a foto pronta e leve. O OV3660 tem 3 MP (2048×1536) contra 2 MP (1600×1200) do OV2640, melhor sensibilidade em pouca luz e a imagem vem **de cabeça para baixo** (o código corrige com `set_vflip`).

Montagem para gravação

O ESP32-CAM não tem chip USB. Ligue um adaptador **USB-serial (FTDI/CH340) configurado em 5 V**:

Adaptador	ESP32-CAM	Observação
TXO	U0R (GPIO 3)	TX de um no RX do outro
RXI	U0T (GPIO 1)	
5V	5V	adaptador bom ou fonte externa

Adaptador	ESP32-CAM	Observação
GND	GND	
—	**GPIO 0 ↔ GND**	**só durante a gravação** (modo flash); depois solte e aperte RST

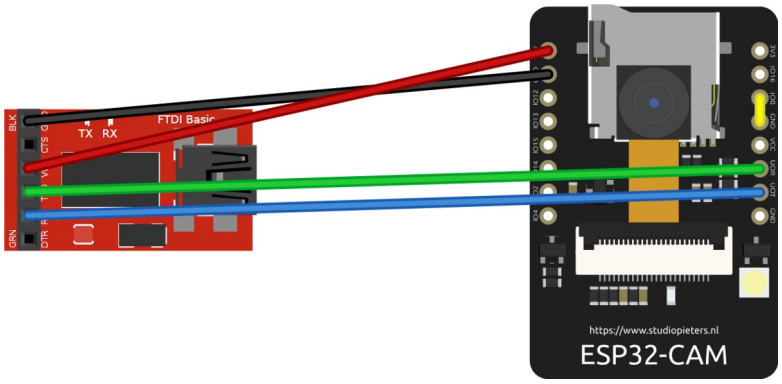


Figura 1 — Aula 1 — ESP32-CAM ligado ao adaptador USB-serial; o fio amarelo IO0–GND só fica durante o upload.

Ritual de gravação

1) IO0 no GND → 2) aperte RST → 3) clique em Upload (segure até aparecer “Connecting…”) → 4) terminou: tire o jumper IO0 → 5) RST de novo. Se der *Failed to connect*: cabo, 5 V fraco, TX/RX trocados ou IO0 solto.

A base **ESP32-CAM-MB** (com micro-USB) faz tudo isso sozinha: escolha “AI Thinker ESP32-CAM” e grave como num DevKit.

Código 1 — foto e vídeo na rede local

O arquivo `camera_aithinker.h` (comum a todos os sketches) configura os 16 pinos da câmera, pede JPEG, usa a PSRAM e aplica os ajustes do OV3660:

camera_aithinker.h

```
/*
  camera_aithinker.h - inicializacao da camera do ESP32-CAM (AI-Thinker) com sensor OV2640 ou OV3660.
  Copie este arquivo para a pasta de cada sketch. Uso:
      #include "camera_aithinker.h"
      void setup() { ... if (!cameraIniciar()) { ... } }

  Pinos da camera no AI-Thinker (fixos, definidos pela placa):
     XCLK 0, SIOD 26, SIOC 27, Y9..Y2 = 35 34 39 36 21 19 18 5, VSYNC 25, HREF 23, PCLK 22, PWDN 32
  Pinos LIVRES para o seu projeto: 12, 13, 14, 15 (se o microSD nao for usado), 2 (SD 1-bit), 4 (flash),
16 e 33 (LED vermelho, ativo em LOW)
*/
#pragma once
#include "esp_camera.h"

#define PWDN_GPIO_NUM   32
#define RESET_GPIO_NUM  -1
#define XCLK_GPIO_NUM    0
#define SIOD_GPIO_NUM   26
#define SIOC_GPIO_NUM   27
#define Y9_GPIO_NUM     35
#define Y8_GPIO_NUM     34
#define Y7_GPIO_NUM     39
#define Y6_GPIO_NUM     36
#define Y5_GPIO_NUM     21
#define Y4_GPIO_NUM     19
#define Y3_GPIO_NUM     18
```

```
#define Y2_GPIO_NUM      5
#define VSYNC_GPIO_NUM  25
#define HREF_GPIO_NUM   23
#define PCLK_GPIO_NUM   22
#define FLASH_GPIO_NUM   4     // LED branco de alta potencia
#define LED_VERMELHO    33     // LED pequeno no verso (LOW = aceso)

const char* cameraNome = "?";

bool cameraIniciar(framesize_t tamanho = FRAMESIZE_SVGA, int qualidade = 12) {
  camera_config_t c = {};
  c.ledc_channel = LEDC_CHANNEL_0;  c.ledc_timer = LEDC_TIMER_0;
  c.pin_d0 = Y2_GPIO_NUM; c.pin_d1 = Y3_GPIO_NUM; c.pin_d2 = Y4_GPIO_NUM; c.pin_d3 = Y5_GPIO_NUM;
  c.pin_d4 = Y6_GPIO_NUM; c.pin_d5 = Y7_GPIO_NUM; c.pin_d6 = Y8_GPIO_NUM; c.pin_d7 = Y9_GPIO_NUM;
  c.pin_xclk = XCLK_GPIO_NUM; c.pin_pclk = PCLK_GPIO_NUM; c.pin_vsync = VSYNC_GPIO_NUM; c.pin_href =
HREF_GPIO_NUM;
  c.pin_sccb_sda = SIOD_GPIO_NUM; c.pin_sccb_scl = SIOC_GPIO_NUM; c.pin_pwdn = PWDN_GPIO_NUM; c.pin_reset
= RESET_GPIO_NUM;
  c.xclk_freq_hz = 20000000;          // 20 MHz para o sensor
  c.pixel_format = PIXFORMAT_JPEG;    // o sensor ja entrega JPEG comprimido
  c.frame_size = tamanho;             // SVGA 800x600 e um bom equilibrio; OV3660 vai ate QXGA 2048x1536
  c.jpeg_quality = qualidade;         // 0 (melhor) .. 63 (pior); 10-12 e bom
  c.fb_count = psramFound() ? 2 : 1;  // 2 buffers na PSRAM = video mais fluido
  c.fb_location = psramFound() ? CAMERA_FB_IN_PSRAM : CAMERA_FB_IN_DRAM;
  c.grab_mode = CAMERA_GRAB_LATEST;
  if (!psramFound()) { c.frame_size = FRAMESIZE_VGA; }

  esp_err_t err = esp_camera_init(&c);
  if (err != ESP_OK) { Serial.printf("Camera falhou: 0x%x (cabo flat? PSRAM? alimentacao 5 V?)\n", err);
return false; }

  sensor_t* s = esp_camera_sensor_get();
  if (s->id.PID == OV3660_PID) {      // ajustes recomendados para o OV3660
    cameraNome = "OV3660";
    s->set_vflip(s, 1);               // a imagem vem de cabeca para baixo neste sensor
    s->set_brightness(s, 1);
    s->set_saturation(s, -2);
  } else if (s->id.PID == OV2640_PID) cameraNome = "OV2640";
  Serial.printf("Camera OK: sensor %s, PSRAM %s\n", cameraNome, psramFound() ? "sim" : "NAO");
  return true;
}
```

01_cam_foto_stream.ino

```
/*
  01 - ESP32-CAM: foto e video ao vivo na rede local
  Placa: "AI Thinker ESP32-CAM" (esp32:esp32:esp32cam)   Sem bibliotecas extras (esp_camera vem no core)
  Arquivo extra na pasta: camera_aithinker.h

  Gravacao: o ESP32-CAM NAO tem USB. Use um adaptador USB-serial (FTDI/CH340) em 5 V:
     adaptador TX -> U0R (GPIO 3)   adaptador RX -> U0T (GPIO 1)   5V -> 5V   GND -> GND
     GPIO 0 ligado ao GND SOMENTE durante a gravacao (modo flash); depois solte e aperte RST.
     (A base "ESP32-CAM-MB" faz isso sozinha, com um micro-USB.)

  O ESP32 cria a rede "Camera-ESP32" (senha 12345678). Abra http://192.168.4.1
     /            pagina com o video ao vivo e botoes
     /capture     uma foto JPEG (abra num navegador, ou use no Python)
     /stream      video MJPEG (na porta 81): http://192.168.4.1:81/stream
     /flash?on=1  liga/desliga o LED de flash (GPIO 4)
     /tamanho?v=N muda a resolucao (5 = QVGA 320x240, 8 = VGA, 9 = SVGA, 10 = XGA, 13 = UXGA 1600x1200)
*/
#include <WiFi.h>
#include "esp_http_server.h"
#include "camera_aithinker.h"

const char* SSID_AP = "Camera-ESP32";
const char* SENHA_AP = "12345678";
httpd_handle_t servidorWeb = NULL, servidorStream = NULL;

const char PAGINA[] = R"HTML(<!DOCTYPE html><html lang="pt-br"><head><meta charset="utf-8"><meta
```

```
name="viewport" content="width=device-width,initial-scale=1">
<title>ESP32-CAM</title><style>body{font-family:sans-serif;background:#111;color:#eee;text-
align:center;margin:0;padding:10px}
img{max-width:100%;border-radius:8px}button{font-size:1em;padding:10px 16px;margin:6px;border:0;border-
radius:8px;background:#448;color:#fff}</style></head>
<body><h2>ESP32-CAM ao vivo</h2><img id="v" src=""><br>
<button onclick="v.src=location.protocol+'//'+location.hostname+':81/stream'">Video</button>
<button onclick="v.src='/capture?t='+Date.now()">Foto</button>
<button onclick="fetch('/flash?on=1')">Flash on</button><button onclick="fetch('/flash?on=0')">Flash
off</button><br>
<button onclick="fetch('/tamanho?v=5')">320x240</button><button
onclick="fetch('/tamanho?v=8')">640x480</button>
<button onclick="fetch('/tamanho?v=9')">800x600</button><button onclick="fetch('/tamanho?
v=13')">1600x1200</button>
<script>v.src=location.protocol+'//'+location.hostname+':81/stream'</script></body></html>)HTML";


// ---------- handlers ----------
static esp_err_t paginaHandler(httpd_req_t* req) {
  httpd_resp_set_type(req, "text/html; charset=utf-8");
  return httpd_resp_send(req, PAGINA, HTTPD_RESP_USE_STRLEN);
}
static esp_err_t captureHandler(httpd_req_t* req) {
  camera_fb_t* fb = esp_camera_fb_get();               // pega um quadro (frame buffer)
  if (!fb) { httpd_resp_send_500(req); return ESP_FAIL; }
  httpd_resp_set_type(req, "image/jpeg");
  httpd_resp_set_hdr(req, "Content-Disposition", "inline; filename=foto.jpg");
  httpd_resp_set_hdr(req, "Access-Control-Allow-Origin", "*");
  esp_err_t r = httpd_resp_send(req, (const char*)fb->buf, fb->len);
  esp_camera_fb_return(fb);                           // devolve o buffer (obrigatorio!)
  return r;
}
static esp_err_t streamHandler(httpd_req_t* req) {    // MJPEG: uma sequencia de JPEGs separados por um
"boundary"
  httpd_resp_set_type(req, "multipart/x-mixed-replace;boundary=frame");
  httpd_resp_set_hdr(req, "Access-Control-Allow-Origin", "*");
  char cabecalho[64];
  while (true) {
    camera_fb_t* fb = esp_camera_fb_get();
    if (!fb) break;
    size_t n = snprintf(cabecalho, sizeof(cabecalho), "\r\n--frame\r\nContent-Type: image/jpeg\r\nContent-
Length: %u\r\n\r\n", fb->len);
    esp_err_t r = httpd_resp_send_chunk(req, cabecalho, n);
    if (r == ESP_OK) r = httpd_resp_send_chunk(req, (const char*)fb->buf, fb->len);
    esp_camera_fb_return(fb);
    if (r != ESP_OK) break;                           // o navegador fechou: para o laco
  }
  return ESP_OK;
}
static int lerArg(httpd_req_t* req, const char* nome) {   // le ?nome=valor da URL
  char url[64], valor[8];
  if (httpd_req_get_url_query_str(req, url, sizeof(url)) == ESP_OK && httpd_query_key_value(url, nome,
valor, sizeof(valor)) == ESP_OK) return atoi(valor);
  return -1;
}
static esp_err_t flashHandler(httpd_req_t* req) {
  int on = lerArg(req, "on");
  if (on >= 0) digitalWrite(FLASH_GPIO_NUM, on ? HIGH : LOW);
  return httpd_resp_send(req, on ? "flash ligado" : "flash desligado", HTTPD_RESP_USE_STRLEN);
}
static esp_err_t tamanhoHandler(httpd_req_t* req) {
  int v = lerArg(req, "v");
  if (v >= 0) esp_camera_sensor_get()->set_framesize(esp_camera_sensor_get(), (framesize_t)v);
  return httpd_resp_send(req, "ok", HTTPD_RESP_USE_STRLEN);
}

void iniciarServidores() {
  httpd_config_t cfg = HTTPD_DEFAULT_CONFIG();
  httpd_uri_t uPagina = { "/", HTTP_GET, paginaHandler, NULL };
  httpd_uri_t uCapture = { "/capture", HTTP_GET, captureHandler, NULL };
  httpd_uri_t uFlash = { "/flash", HTTP_GET, flashHandler, NULL };
  httpd_uri_t uTam = { "/tamanho", HTTP_GET, tamanhoHandler, NULL };
  if (httpd_start(&servidorWeb, &cfg) == ESP_OK) {
    httpd_register_uri_handler(servidorWeb, &uPagina); httpd_register_uri_handler(servidorWeb, &uCapture);
    httpd_register_uri_handler(servidorWeb, &uFlash);  httpd_register_uri_handler(servidorWeb, &uTam);
  }
  cfg.server_port = 81; cfg.ctrl_port = 32769;        // o stream fica num servidor separado (porta 81)
```

```
para nao travar os outros pedidos
  httpd_uri_t uStream = { "/stream", HTTP_GET, streamHandler, NULL };
  if (httpd_start(&servidorStream, &cfg) == ESP_OK) httpd_register_uri_handler(servidorStream, &uStream);
}

void setup() {
  Serial.begin(115200);
  pinMode(FLASH_GPIO_NUM, OUTPUT); digitalWrite(FLASH_GPIO_NUM, LOW);
  if (!cameraIniciar(FRAMESIZE_SVGA, 12)) { while (true) delay(1000); }
  WiFi.softAP(SSID_AP, SENHA_AP);
  iniciarServidores();
  Serial.print("Rede "); Serial.print(SSID_AP); Serial.print(" -> http://");
Serial.println(WiFi.softAPIP());
  Serial.println("Stream: http://192.168.4.1:81/stream   Foto: http://192.168.4.1/capture");
}

void loop() { delay(100); }   // os servidores rodam em tarefas proprias; o loop fica livre
```

Entenda o código

- `esp_camera_fb_get()` devolve um **frame buffer** (`fb->buf`, `fb->len`) já em JPEG; `esp_camera_fb_return()` **devolve** o buffer — esquecer isso trava a câmera depois de 2 quadros.
- **MJPEG** = *Motion JPEG*: o servidor manda um JPEG atrás do outro, separados por `--frame`, numa resposta que nunca termina (`multipart/x-mixed-replace`). O navegador troca a imagem a cada quadro — é o “vídeo” mais simples que existe.
- Dois servidores HTTP: o stream fica na **porta 81** para não bloquear `/capture` e `/flash` na 80.
- `set_framesize` troca a resolução ao vivo; `jpeg_quality` 10–12 é o equilíbrio entre nitidez e tamanho.
- `psramFound()`: sem PSRAM (algumas placas), fica em VGA e 1 buffer.

Experimente

1. Abra `http://192.168.4.1:81/stream` direto no navegador e depois `/capture`: salve a foto e veja o tamanho em kB em cada resolução.
2. Meça os quadros por segundo em 320×240, 800×600 e 1600×1200 (conte em 10 s). Por que cai? (mais bytes por quadro no Wi-Fi + tempo de compressão)
3. Ligue o flash e fotografe no escuro; depois tampe a lente: o sensor “ganha” ruído (ganho automático).
4. Aponte para uma lâmpada fluorescente: vê faixas? É o *rolling shutter* + 60 Hz.

Desafios

1. **1.1 Foto por botão** — botão no GPIO 13 (INPUT_PULLUP) dispara o flash e uma foto; o LED vermelho da placa (GPIO 33, ativo em LOW) pisca ao fotografar.
2. **1.2 Modo noturno automático** — leia o brilho médio do quadro (a biblioteca dá a imagem só em JPEG; use `s->set_special_effect` / ou estime pelo tamanho do JPEG: escuro = arquivo pequeno) e ligue o flash quando escurecer.
3. **1.3 Pan digital** — botões na página que mudam `set_hmirror`/`set_vflip` e o zoom (`FRAMESIZE` menor recortada: `set_res_raw`).

4. **1.4 Contador de espectadores** — mostre na página quantos clientes estão vendo o stream (conte handlers ativos com uma variável global).
5. **1.5 Buzzer de "ao vivo"** — bipe curto (GPIO 14) sempre que alguém abrir o stream — como a luz vermelha de uma câmera de estúdio.

Aula 2 — Câmera de segurança: PIR, microSD e galeria

O que você vai aprender

- Como o sensor **PIR** detecta pessoas (infravermelho + lente de Fresnel).
- Gravar arquivos no microSD (`SD_MMC`) com nomes por data/hora (NTP).
- Uma **galeria web** que lista, mostra e apaga fotos do cartão.
- Lógica de eventos: borda do PIR, intervalo mínimo, timelapse.

A física do PIR

Todo corpo quente emite **infravermelho** (~10 µm para 37 °C). O PIR tem dois elementos piroelétricos lado a lado: quando uma fonte de calor **se move** de um para o outro, a diferença entre eles gera um pulso. A cúpula branca é uma **lente de Fresnel** com dezenas de facetas que dividem o campo em "fatias" — por isso ele detecta *movimento* de gente, não presença parada, e não vê através de vidro (o vidro bloqueia IR distante). O HC-SR501 tem dois potenciômetros (sensibilidade/alcance até 7 m e tempo de saída 3 s–5 min) e precisa de ~60 s para estabilizar ao ligar.

Montagem

Componente	ESP32-CAM	Observação
PIR VCC / GND / OUT	5V / GND / **GPIO 13**	OUT sai em 3,3 V
LED de status	GPIO 12 → 220 Ω → LED → GND	GPIO 12 deve estar LOW no boot: LED para o GND é seguro
microSD	slot da placa	FAT32, ≤ 32 GB, modo 1-bit no código
Flash	GPIO 4	livre porque o SD está em 1-bit

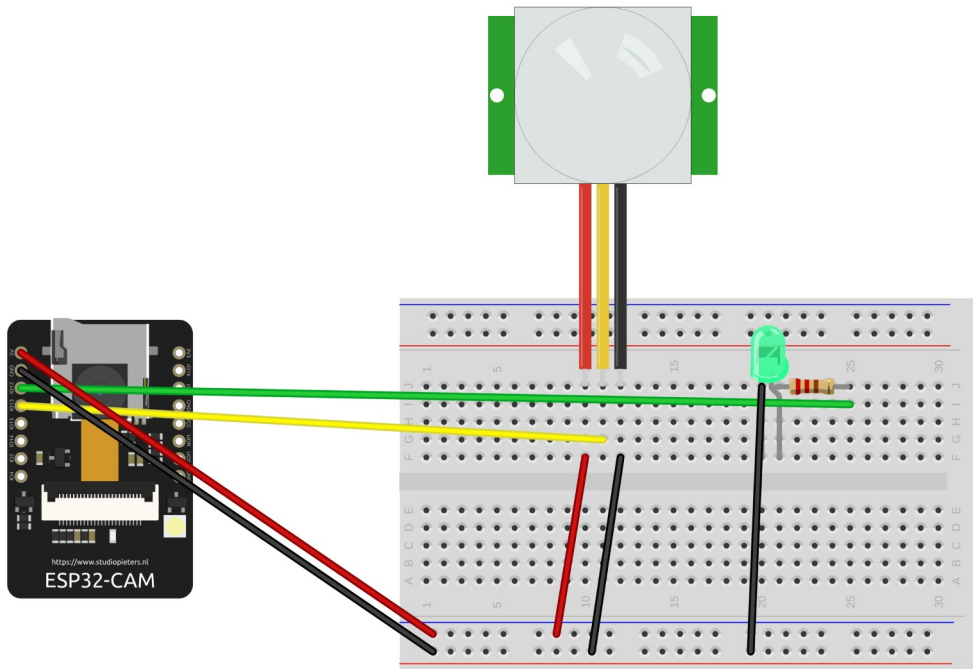


Figura 2 — Aula 2 — PIR no GPIO 13 e LED de status no GPIO 12; alimentação 5 V pela trilha da protoboard.

Código 2 — segurança com SD e galeria

02_cam_seguranca_sd.ino

```
/*
  02 - Camera de seguranca: sensor de presenca (PIR) + gravacao no microSD + galeria web
  Placa: AI Thinker ESP32-CAM      Arquivo extra: camera_aithinker.h
  Bibliotecas: SD_MMC, FS (vem no core)

  Ligacoes:
    PIR HC-SR501: VCC -> 5V   GND -> GND   OUT -> GPIO 13   (o PIR sai em 3,3 V, ok)
    LED de status (verde) -> resistor 220 -> GPIO 12 -> GND  (GPIO 12 deve ficar em LOW no boot: LED para
o GND e seguro)
    microSD no slot (formatado em FAT32, ate 32 GB)
    Flash: GPIO 4 (LED da propria placa)

  O microSD e usado em modo 1 bit (GPIO 2, 14, 15) para liberar os GPIO 4, 12 e 13.
  Cada movimento: liga o flash (se escuro/opcao), tira uma foto, grava /fotos/AAAA-MM-DD_HH-MM-SS.jpg.
  Data/hora: se MODO_AP = false, o ESP32 entra na sua rede e acerta o relogio pela internet (NTP);
  em modo AP nao ha internet: os nomes usam o tempo desde que ligou.
  Pagina: /  (ao vivo + galeria)   /lista  (JSON)   /foto?n=nome   /apagar?n=nome   /modo?timelapse=SEG
*/
#include <WiFi.h>
#include <time.h>
#include "FS.h"
#include "SD_MMC.h"
#include "esp_http_server.h"
#include "camera_aithinker.h"

const bool MODO_AP = true;
const char* SSID_AP = "Camera-ESP32";  const char* SENHA_AP = "12345678";
const char* SSID_REDE = "NOME-DA-REDE"; const char* SENHA_REDE = "SENHA-DA-REDE";
const int PINO_PIR = 13, PINO_LED = 12;
const bool USAR_FLASH = true;
const unsigned long INTERVALO_MIN_MS = 5000;   // nao tira 2 fotos em menos de 5 s

httpd_handle_t servidorWeb = NULL, servidorStream = NULL;
unsigned long ultimaFoto = 0, timelapseSeg = 0, ultimoTimelapse = 0;
int totalFotos = 0;
bool sdOk = false;

// ---------- utilidades ----------
String nomeArquivo() {
  struct tm t;
  char nome[48];
  if (getLocalTime(&t, 10)) strftime(nome, sizeof(nome), "/fotos/%Y-%m-%d_%H-%M-%S.jpg", &t);
  else snprintf(nome, sizeof(nome), "/fotos/t%08lu.jpg", millis() / 1000);
  return String(nome);
}
bool tirarFoto(const char* motivo) {
  if (USAR_FLASH) { digitalWrite(FLASH_GPIO_NUM, HIGH); delay(120); }
  camera_fb_t* fb = esp_camera_fb_get();
  digitalWrite(FLASH_GPIO_NUM, LOW);
  if (!fb) { Serial.println("sem quadro"); return false; }
  bool ok = false;
  if (sdOk) {
    String nome = nomeArquivo();
    File f = SD_MMC.open(nome, FILE_WRITE);
    if (f) { f.write(fb->buf, fb->len); f.close(); ok = true; totalFotos++; Serial.printf("[%s] gravada %s
(%u bytes)\n", motivo, nome.c_str(), fb->len); }
    else Serial.println("erro ao abrir arquivo no SD");
  }
  esp_camera_fb_return(fb);
  return ok;
}

// ---------- web ----------
const char PAGINA[] = R"HTML(<!DOCTYPE html><html lang="pt-br"><head><meta charset="utf-8"><meta
name="viewport" content="width=device-width,initial-scale=1">
<title>Camera de seguranca</title><style>body{font-family:sans-
serif;background:#111;color:#eee;margin:0;padding:10px;text-align:center}
img{max-width:100%;border-radius:8px}.g{display:flex;flex-wrap:wrap;gap:8px;justify-content:center}.g
div{background:#222;padding:6px;border-radius:8px;font-size:.8em}
.g img{width:160px;height:120px;object-fit:cover}button{font-size:.9em;padding:8px
12px;margin:4px;border:0;border-radius:8px;background:#448;color:#fff}
#st{color:#8f8}</style></head><body><h2>Camera de seguranca</h2>
```

```
<img id="v"><p id="st"></p>
<button onclick="fetch('/foto_agora').then(lista)">Tirar foto agora</button>
<button onclick="fetch('/modo?timelapse=30').then(lista)">Timelapse 30 s</button><button
onclick="fetch('/modo?timelapse=0').then(lista)">Parar timelapse</button>
<h3>Galeria (SD)</h3><div class="g" id="g"></div>
<script>v.src=location.protocol+'//'+location.hostname+':81/stream';
async function lista(){const d=await (await fetch('/lista')).json();st.textContent='fotos: '+d.total+' |
movimento: '+(d.pir?'SIM':'nao')+' | timelapse: '+d.timelapse+' s';
g.innerHTML=d.fotos.slice().reverse().map(n=>'<div><a href="/foto?n='+n+'" target="_blank"><img
src="/foto?n='+n+'"></a><br>'+n+' <button
onclick="fetch(\'/apagar?n='+n+'\').then(lista)">x</button></div>').join('')}
lista();setInterval(lista,10000)</script></body></html>)HTML";

static esp_err_t paginaHandler(httpd_req_t* req) { httpd_resp_set_type(req, "text/html; charset=utf-8");
return httpd_resp_send(req, PAGINA, HTTPD_RESP_USE_STRLEN); }
static esp_err_t streamHandler(httpd_req_t* req) {
  httpd_resp_set_type(req, "multipart/x-mixed-replace;boundary=frame");
  char cab[64];
  while (true) {
    camera_fb_t* fb = esp_camera_fb_get(); if (!fb) break;
    size_t n = snprintf(cab, sizeof(cab), "\r\n--frame\r\nContent-Type: image/jpeg\r\nContent-Length:
%u\r\n\r\n", fb->len);
    esp_err_t r = httpd_resp_send_chunk(req, cab, n);
    if (r == ESP_OK) r = httpd_resp_send_chunk(req, (const char*)fb->buf, fb->len);
    esp_camera_fb_return(fb);
    if (r != ESP_OK) break;
  }
  return ESP_OK;
}
static String argStr(httpd_req_t* req, const char* nome) {
  char url[128], v[64];
  if (httpd_req_get_url_query_str(req, url, sizeof(url)) == ESP_OK && httpd_query_key_value(url, nome, v,
sizeof(v)) == ESP_OK) return String(v);
  return "";
}
static esp_err_t listaHandler(httpd_req_t* req) {
  String json = "{\"total\":" + String(totalFotos) + ",\"pir\":" + (digitalRead(PINO_PIR) ? "true" :
"false") + ",\"timelapse\":" + String(timelapseSeg) + ",\"fotos\":[";
  if (sdOk) {
    File dir = SD_MMC.open("/fotos"); File f = dir.openNextFile(); bool primeiro = true; int n = 0;
    while (f && n < 60) { if (!f.isDirectory()) { if (!primeiro) json += ","; json += "\"" +
String(f.name()) + "\""; primeiro = false; n++; } f = dir.openNextFile(); }
  }
  json += "]}";
  httpd_resp_set_type(req, "application/json");
  return httpd_resp_send(req, json.c_str(), json.length());
}
static esp_err_t fotoHandler(httpd_req_t* req) {          // serve um JPEG do SD
  String n = argStr(req, "n"); File f = SD_MMC.open("/fotos/" + n);
  if (!f) { httpd_resp_send_404(req); return ESP_FAIL; }
  httpd_resp_set_type(req, "image/jpeg");
  char buf[2048]; size_t lidos;
  while ((lidos = f.read((uint8_t*)buf, sizeof(buf))) > 0) httpd_resp_send_chunk(req, buf, lidos);
  f.close(); httpd_resp_send_chunk(req, NULL, 0);
  return ESP_OK;
}
static esp_err_t apagarHandler(httpd_req_t* req) { String n = argStr(req, "n"); SD_MMC.remove("/fotos/" +
n); return httpd_resp_send(req, "ok", HTTPD_RESP_USE_STRLEN); }
static esp_err_t fotoAgoraHandler(httpd_req_t* req) { tirarFoto("web"); return httpd_resp_send(req, "ok",
HTTPD_RESP_USE_STRLEN); }
static esp_err_t modoHandler(httpd_req_t* req) { timelapseSeg = argStr(req, "timelapse").toInt(); return
httpd_resp_send(req, "ok", HTTPD_RESP_USE_STRLEN); }

void iniciarServidores() {
  httpd_config_t cfg = HTTPD_DEFAULT_CONFIG(); cfg.max_uri_handlers = 10;
  httpd_uri_t us[] = { {"/", HTTP_GET, paginaHandler, NULL}, {"/lista", HTTP_GET, listaHandler, NULL},
{"/foto", HTTP_GET, fotoHandler, NULL},
                       {"/apagar", HTTP_GET, apagarHandler, NULL}, {"/foto_agora", HTTP_GET,
fotoAgoraHandler, NULL}, {"/modo", HTTP_GET, modoHandler, NULL} };
  if (httpd_start(&servidorWeb, &cfg) == ESP_OK) for (auto& u : us)
httpd_register_uri_handler(servidorWeb, &u);
  cfg.server_port = 81; cfg.ctrl_port = 32769;
  httpd_uri_t uStream = { "/stream", HTTP_GET, streamHandler, NULL };
  if (httpd_start(&servidorStream, &cfg) == ESP_OK) httpd_register_uri_handler(servidorStream, &uStream);
}
```

```
void setup() {
  Serial.begin(115200);
  pinMode(FLASH_GPIO_NUM, OUTPUT); digitalWrite(FLASH_GPIO_NUM, LOW);
  pinMode(PINO_PIR, INPUT); pinMode(PINO_LED, OUTPUT);
  if (!cameraIniciar(FRAMESIZE_SVGA, 12)) { while (true) delay(1000); }

  sdOk = SD_MMC.begin("/sdcard", true);              // true = modo 1 bit (libera GPIO 4, 12, 13)
  if (sdOk) { SD_MMC.mkdir("/fotos"); Serial.printf("SD OK: %llu MB\n", SD_MMC.cardSize() / (1024 *
1024)); }
  else Serial.println("SD nao encontrado: fotos nao serao gravadas");

  if (MODO_AP) { WiFi.softAP(SSID_AP, SENHA_AP); Serial.print("http://"); Serial.println(WiFi.softAPIP());
}
  else {
    WiFi.begin(SSID_REDE, SENHA_REDE);
    while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
    Serial.print("\nhttp://"); Serial.println(WiFi.localIP());
    configTime(-3 * 3600, 0, "pool.ntp.org");        // horario de Brasilia
  }
  iniciarServidores();
  digitalWrite(PINO_LED, HIGH);
  Serial.println("O PIR precisa de ~60 s para estabilizar apos ligar.");
}

void loop() {
  // 1) movimento (borda de subida do PIR) com intervalo minimo
  static bool pirAntes = false;
  bool pir = digitalRead(PINO_PIR);
  if (pir && !pirAntes && millis() - ultimaFoto > INTERVALO_MIN_MS) {
    ultimaFoto = millis();
    digitalWrite(PINO_LED, LOW); tirarFoto("PIR"); digitalWrite(PINO_LED, HIGH);
  }
  pirAntes = pir;
  // 2) timelapse
  if (timelapseSeg > 0 && millis() - ultimoTimelapse >= timelapseSeg * 1000UL) { ultimoTimelapse =
millis(); tirarFoto("timelapse"); }
  delay(50);
}
```

Entenda o código

- `SD_MMC.begin("/sdcard", true)`: o `true` é o **modo 1-bit** — mais lento (~1 MB/s), mas libera GPIO 4, 12 e 13. Sem ele o flash e o PIR conflitariam com o cartão.
- `nomeArquivo()` usa `getLocalTime` (NTP, só com internet) e cai para “t+segundos” em modo AP: um arquivo **nunca** deve sobrescrever outro.
- O PIR é tratado na **borda de subida** com intervalo mínimo de 5 s — a mesma ideia do botão com debounce.
- A galeria é `/lista` (JSON com os nomes) + `/foto?n=` (o ESP32 lê o arquivo em pedaços de 2 kB e manda `chunk` a `chunk`).
- Timelapse: `/modo?timelapse=30` → uma foto a cada 30 s; junte-as no PC com o OpenCV (Aula 4) e vira vídeo.

Experimente

1. Ajuste o pot de sensibilidade do PIR e meça o alcance com passos de 1 m.
2. Fique parado 30 s na frente: o PIR dispara? Levante um braço.
3. Tire o cartão e abra as fotos no PC: os nomes contam a história do dia.

4. Timelapse de 10 s durante 10 min apontado para a janela; monte o vídeo (desafio 4.3).

Desafios

1. **2.1 Alarme sonoro** — buzzer (GPIO 14) toca 3 bipes ao detectar movimento e o LED RGB fica vermelho por 10 s; botão “armar/desarmar” no GPIO 15 com LED verde/vermelho de estado.
2. **2.2 Rajada** — em vez de 1 foto, 3 fotos com 1 s de intervalo por evento (o ladrão não fica parado).
3. **2.3 Cota do cartão** — se houver mais de 200 fotos, apague a mais antiga antes de gravar (rotação).
4. **2.4 Horário de vigilância** — com NTP, só fotografa entre 18h e 7h (modo estação); fora disso o LED azul indica “desarmado”.
5. **2.5 Contador de passagens** — conte eventos por hora e mostre na página um histograma simples (24 barras) — quando o corredor é mais movimentado?

Aula 3 — Câmera na internet: Google Drive e Telegram

O que você vai aprender

- A diferença entre **rede local** e **internet**, e por que o `/stream` não sai da sala (NAT, IP privado).
- Mandar um arquivo para o **Google Drive** por HTTPS usando um **Apps Script** como "porteiro".
- Enviar uma foto pelo **Telegram** (`sendPhoto`, multipart).
- Base64, cooldown e credenciais.

Por que o vídeo ao vivo não sai da rede local

O ESP32 tem um IP **privado** (192.168.x.x) atrás do roteador (**NAT**): a internet não consegue iniciar uma conexão até ele. Para ver o stream de fora existem três caminhos: (1) **encaminhamento de porta** no roteador (expõe a câmera à internet — perigoso sem senha); (2) um **túnel** como Tailscale ou ngrok rodando num PC da mesma rede; (3) o caminho "empurrado": o ESP32 **envia** as imagens para um serviço na nuvem (Drive, Telegram) e você as vê de qualquer lugar. Faremos o (3): é seguro, não precisa de roteador configurado, e com uma foto a cada N segundos vira um "quase ao vivo".

Preparação (com o professor)

1. **Google Drive:** crie a pasta `ESP32-CAM`, publique o `Code.gs` como *App da Web* (executar como você; acesso: qualquer pessoa) e copie a URL `/exec` para `URL_APPS_SCRIPT`.
2. **Telegram:** fale com `@BotFather` → `/newbot` → copie o **token**; fale com `@userinfobot` → copie o seu **chat_id**; mande "oi" para o seu bot uma vez.
3. Preencha Wi-Fi (com internet), token, chat_id e URL no sketch 03.

apps_script/Code.gs (Google Apps Script)

```
/*
  Google Apps Script - recebe a foto do ESP32-CAM e grava no Google Drive.

  Como publicar (uma vez, com a conta Google do laboratorio):
    1. script.google.com -> Novo projeto -> cole este codigo -> salve.
    2. Crie no Drive uma pasta "ESP32-CAM" e copie o ID dela (parte final da URL da pasta) em PASTA_ID.
    3. Implantar -> Nova implantacao -> Tipo: "App da Web" -> Executar como: EU -> Quem tem acesso:
"Qualquer pessoa".
    4. Autorize o acesso ao Drive. Copie a URL que termina em /exec para o sketch (URL_APPS_SCRIPT).
    5. Cada vez que mudar o codigo: Implantar -> Gerenciar implantacoes -> editar -> nova versao.

  O ESP32 faz POST com "motivo=texto&imagem=<base64 do JPEG>". Testar no navegador (GET) tambem funciona.
*/
const PASTA_ID = "COLE-AQUI-O-ID-DA-PASTA";

function doPost(e) {
  try {
    const motivo = (e.parameter.motivo || "foto").replace(/[^\w-]/g, "_");
    const base64 = e.parameter.imagem;
    if (!base64) return ContentService.createTextOutput("faltou o parametro imagem");
    const bytes = Utilities.base64Decode(base64);
    const agora = Utilities.formatDate(new Date(), "America/Recife", "yyyy-MM-dd_HH-mm-ss");
    const blob = Utilities.newBlob(bytes, "image/jpeg", agora + "_" + motivo + ".jpg");
    const pasta = DriveApp.getFolderById(PASTA_ID);
    const arquivo = pasta.createFile(blob);
    // mantem sempre uma copia chamada "ultima.jpg" = a foto mais recente (para o "quase ao vivo")
    const antigas = pasta.getFilesByName("ultima.jpg");
    while (antigas.hasNext()) antigas.next().setTrashed(true);
    pasta.createFile(blob.setName("ultima.jpg"));
    return ContentService.createTextOutput("ok " + arquivo.getName());
  } catch (erro) {
    return ContentService.createTextOutput("erro: " + erro);
```

```
  }
}

function doGet(e) {           // abrir a URL no navegador mostra que o script esta no ar
  return ContentService.createTextOutput("ESP32-CAM Drive uploader ativo. Use POST com motivo e imagem
(base64).");
}
```

Código 3 — Drive + Telegram

03_cam_drive_telegram.ino

```
/*
  03 - Pela internet: foto para o Google Drive (Apps Script) e alerta com foto no Telegram
  Placa: AI Thinker ESP32-CAM      Arquivo extra: camera_aithinker.h      Sem bibliotecas extras

  Ligacoes: iguais ao sketch 02 (PIR no GPIO 13, LED no 12). SD opcional.
  O ESP32 entra na rede Wi-Fi COM INTERNET. A cada movimento (ou a cada INTERVALO_S segundos):
    1) tira a foto
    2) manda para o Google Drive: POST HTTPS para um "Web App" do Google Apps Script (veja
apps_script/Code.gs),
       com a imagem em BASE64 no corpo. O script cria o arquivo na sua pasta do Drive.
    3) manda para o Telegram: POST multipart para api.telegram.org/bot<TOKEN>/sendPhoto (a foto chega no
celular).
  "Ao vivo pela internet": o /stream so funciona na rede local. Para ver de fora, o guia explica as opcoes
  (Tailscale/ngrok/port-forward) - aqui fazemos "quase ao vivo": uma foto a cada N segundos no Drive.

  Preparacao: apps_script/Code.gs (Drive) e um bot do Telegram (@BotFather -> token; seu chat_id via
@userinfobot).
*/
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include <base64.h>
#include "camera_aithinker.h"

// ---------- preencha ----------
const char* WIFI_SSID = "NOME-DA-REDE";
const char* WIFI_SENHA = "SENHA-DA-REDE";
const char* URL_APPS_SCRIPT = "https://script.google.com/macros/s/AKfycb.../exec";   // URL do Web App
implantado
const char* TELEGRAM_TOKEN = "123456789:ABCDEF...";
const char* TELEGRAM_CHAT_ID = "123456789";
const int PINO_PIR = 13, PINO_LED = 12;
const unsigned long INTERVALO_S = 0;        // 0 = so por movimento; 60 = tambem uma foto por minuto
("quase ao vivo")
const unsigned long COOLDOWN_MS = 30000;    // no maximo 1 envio por movimento a cada 30 s

unsigned long ultimoEnvio = 0, ultimoPeriodico = 0;

// ---------- Google Drive via Apps Script ----------
bool enviarDrive(camera_fb_t* fb, const char* motivo) {
  WiFiClientSecure cliente; cliente.setInsecure();
  HTTPClient http;
  http.begin(cliente, URL_APPS_SCRIPT);
  http.setFollowRedirects(HTTPC_STRICT_FOLLOW_REDIRECTS);     // o Apps Script responde com redirect 302
  http.addHeader("Content-Type", "application/x-www-form-urlencoded");
  String corpo = "motivo=" + String(motivo) + "&imagem=" + base64::encode(fb->buf, fb->len);   // JPEG
vira texto base64 (+33 %)
  int codigo = http.POST(corpo);
  String resp = http.getString(); http.end();
  Serial.printf("Drive: HTTP %d %s\n", codigo, resp.substring(0, 80).c_str());
  return codigo == 200;
}

// ---------- Telegram sendPhoto (multipart/form-data) ----------
bool enviarTelegram(camera_fb_t* fb, const char* legenda) {
  WiFiClientSecure cliente; cliente.setInsecure();
  if (!cliente.connect("api.telegram.org", 443)) { Serial.println("Telegram: nao conectou"); return false;
```

```
}
  String limite = "----ESP32CAM";
  String inicio = "--" + limite + "\r\nContent-Disposition: form-data; name=\"chat_id\"\r\n\r\n" +
TELEGRAM_CHAT_ID + "\r\n" +
                  "--" + limite + "\r\nContent-Disposition: form-data; name=\"caption\"\r\n\r\n" + legenda
+ "\r\n" +
                  "--" + limite + "\r\nContent-Disposition: form-data; name=\"photo\";
filename=\"foto.jpg\"\r\nContent-Type: image/jpeg\r\n\r\n";
  String fim = "\r\n--" + limite + "--\r\n";
  cliente.printf("POST /bot%s/sendPhoto HTTP/1.1\r\nHost: api.telegram.org\r\nContent-Type:
multipart/form-data; boundary=%s\r\nContent-Length: %u\r\nConnection: close\r\n\r\n",
                 TELEGRAM_TOKEN, limite.c_str(), inicio.length() + fb->len + fim.length());
  cliente.print(inicio);
  for (size_t i = 0; i < fb->len; i += 1024) cliente.write(fb->buf + i, min((size_t)1024, fb->len - i));
// envia o JPEG em pedacos
  cliente.print(fim);
  unsigned long t0 = millis(); String linha;
  while (cliente.connected() && millis() - t0 < 10000) { linha = cliente.readStringUntil('\n'); if
(linha.startsWith("HTTP/")) break; }
  cliente.stop();
  Serial.println("Telegram: " + linha);
  return linha.indexOf("200") > 0;
}

void enviarFoto(const char* motivo) {
  digitalWrite(PINO_LED, LOW);
  camera_fb_t* fb = esp_camera_fb_get();
  if (!fb) { Serial.println("sem quadro"); return; }
  Serial.printf("Foto de %u bytes (%s). Enviando...\n", fb->len, motivo);
  enviarDrive(fb, motivo);
  enviarTelegram(fb, (String("ESP32-CAM: ") + motivo).c_str());
  esp_camera_fb_return(fb);
  digitalWrite(PINO_LED, HIGH);
}

void setup() {
  Serial.begin(115200);
  pinMode(PINO_PIR, INPUT); pinMode(PINO_LED, OUTPUT);
  if (!cameraIniciar(FRAMESIZE_VGA, 12)) { while (true) delay(1000); }   // VGA: ~30 kB por foto, rapido
de enviar
  WiFi.begin(WIFI_SSID, WIFI_SENHA);
  Serial.print("Wi-Fi"); while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  Serial.print("\nIP "); Serial.println(WiFi.localIP());
  digitalWrite(PINO_LED, HIGH);
  enviarFoto("teste ao ligar");
}

void loop() {
  static bool pirAntes = false;
  bool pir = digitalRead(PINO_PIR);
  if (pir && !pirAntes && millis() - ultimoEnvio > COOLDOWN_MS) { ultimoEnvio = millis();
enviarFoto("movimento"); }
  pirAntes = pir;
  if (INTERVALO_S > 0 && millis() - ultimoPeriodico >= INTERVALO_S * 1000UL) { ultimoPeriodico = millis();
enviarFoto("periodica"); }
  delay(50);
}
```

Entenda o código

- **Drive:** o ESP32 faz um `POST` com `imagem=<base64>`. Base64 transforma os bytes do JPEG em texto (26 kB viram 35 kB). O Apps Script decodifica (`Utilities.base64Decode`), cria o arquivo na pasta e ainda mantém uma cópia `ultima.jpg` — abra-a no Drive do celular e atualize: é o “quase ao vivo”.
- `setFollowRedirects`: o Google responde `302` e redireciona para outro domínio — sem seguir o redirect o envio “falha” com 302.

- **Telegram:** `sendPhoto` exige `multipart/form-data`: o corpo tem “partes” separadas por um *boundary*; montamos o cabeçalho de cada parte à mão e mandamos o JPEG em pedaços de 1 kB. Abra o Serial: `HTTP/1.1 200 OK`.
- `COOLDOWN_MS = 30 s` e `INTERVALO_S` (0 = só por movimento; 60 = uma foto por minuto).
- Tudo em HTTPS (`WiFiClientSecure`, `setInsecure()`): criptografado, sem verificar o certificado — o guia do professor discute.

Experimente

1. Abra a URL do Apps Script no navegador: deve responder “ativo”.
2. Passe na frente do PIR: em ~5 s a foto aparece no Drive **e** no Telegram. Meça o tempo total e o tamanho do JPEG no Serial.
3. Mude para `FRAMESIZE_SVGA`: o envio demora quanto a mais?
4. Coloque `INTERVALO_S = 30` e acompanhe `ultima.jpg` pelo celular fora da escola.

Desafios

1. **3.1 Comando de volta** — o Telegram também recebe: use `getUpdates` para ler mensagens; `foto` → manda uma foto; `luz` → liga o flash; `led vermelho/verde` → LED RGB. (O ESP32 pergunta ao servidor a cada 2 s — *polling*, como a página do sensor.)
2. **3.2 Alerta com regra** — só envia se houver **3 detecções em 1 min** (evita gato/cortina); LED amarelo pisca enquanto “contando”.
3. **3.3 Relatório diário** — às 18h (NTP) envia um único e-mail/Telegram com o total de eventos do dia (reuse o SMTP da aula de distância).
4. **3.4 Nome com local** — inclua no nome do arquivo o nome da sala (constante) e no Drive crie uma subpasta por dia (`getFoldersByName` no Apps Script).
5. **3.5 Ao vivo de verdade** — com o professor: rode um túnel (Tailscale) num PC da rede e acesse o `/stream` de fora. Discuta os riscos de expor uma câmera.

Aula 4 — Python: o que um computador vê numa imagem

O que você vai aprender

- Uma imagem é uma **matriz de números**; cor é um trio (B, G, R).
- As etapas clássicas: cinza → bordas → máscara de cor → contornos → detecção de padrões (Haar).
- Ler o stream ou o `/capture` do ESP32-CAM no PC com OpenCV, salvar fotos e gravar vídeo.
- A diferença entre **visão clássica** (regras) e **aprendizado de máquina** (modelos treinados).

Instalação

terminal

```
pip install opencv-python mediapipe requests numpy
# no laboratório (pasta python-dados): uv add opencv-python mediapipe requests
# modelos (pasta python/modelos): gesture_recognizer.task, hand_landmarker.task,
haarcascade_frontalface_default.xml
```

Script 1 — o stream no PC

ver_stream.py

```
"""Ver o video do ESP32-CAM no PC com OpenCV (e salvar fotos/videos).

Uso:
    python ver_stream.py                       # stream padrao http://192.168.4.1:81/stream
    python ver_stream.py http://192.168.1.50:81/stream
Teclas na janela:  f = salvar foto   g = comecar/parar gravacao (.avi)   q = sair

Ideia: o stream MJPEG e "uma foto atras da outra"; cv2.VideoCapture entende esse formato
como se fosse uma webcam. Cada quadro e uma matriz numpy (altura x largura x 3 cores BGR).
"""
import sys, time, cv2

URL = sys.argv[1] if len(sys.argv) > 1 else "http://192.168.4.1:81/stream"

cap = cv2.VideoCapture(URL)
if not cap.isOpened():
    print("nao abriu o stream:", URL, "\nConectado na rede da camera? O ESP32 esta ligado?"); sys.exit(1)

gravador = None
t0, quadros = time.time(), 0
print("f = foto, g = gravar/parar, q = sair")
while True:
    ok, quadro = cap.read()                  # quadro: numpy array HxWx3
    if not ok: print("perdi o stream, tentando de novo..."); time.sleep(0.5); cap = cv2.VideoCapture(URL);
continue
    quadros += 1
    fps = quadros / (time.time() - t0)
    h, w = quadro.shape[:2]
    cv2.putText(quadro, f"{w}x{h}  {fps:.1f} fps" + ("  REC" if gravador else ""), (10, 25),
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255) if gravador else (0, 255, 0), 2)
    if gravador: gravador.write(quadro)
    cv2.imshow("ESP32-CAM", quadro)
    k = cv2.waitKey(1) & 0xFF
    if k == ord("q"): break
    if k == ord("f"):
        nome = time.strftime("foto_%Y%m%d_%H%M%S.jpg"); cv2.imwrite(nome, quadro); print("salva", nome)
    if k == ord("g"):
        if gravador: gravador.release(); gravador = None; print("gravacao parada")
        else:
            nome = time.strftime("video_%Y%m%d_%H%M%S.avi")
            gravador = cv2.VideoWriter(nome, cv2.VideoWriter_fourcc(*"MJPG"), max(5, int(fps)), (w, h));
print("gravando", nome)
```

```
if gravador: gravador.release()
cap.release(); cv2.destroyAllWindows()
```

- `cv2.VideoCapture(url)` entende MJPEG como se fosse uma webcam. `cap.read()` devolve um `numpy.ndarray` **altura × largura × 3** de inteiros 0–255.
- `quadro[0, 0]` é o pixel do canto: três números **B, G, R** (o OpenCV usa essa ordem, ao contrário de quase todo mundo).
- `VideoWriter` grava um `.avi` MJPEG — é o mesmo formato que o ESP32 manda, só que em arquivo.

Script 2 — anatomia de uma imagem

reconhecer_imagem.py

```
"""Aula de visao computacional: o que um computador "ve" numa imagem, passo a passo.

Uso:
    python reconhecer_imagem.py foto.jpg              # analisa uma foto
    python reconhecer_imagem.py http://192.168.4.1/capture   # pega uma foto do ESP32-CAM
    python reconhecer_imagem.py --webcam               # usa a webcam do PC ao vivo
    python reconhecer_imagem.py --demo                 # gera uma imagem de teste (sem hardware)

Abre 5 janelas: original, tons de cinza, bordas (Canny), mascara de cor (vermelho) e rostos (Haar).
Cada uma e uma etapa classica: pixel -> cinza -> bordas -> regioes de cor -> padroes (cascata).
"""
import sys, cv2, numpy as np

def carregar(origem):
    if origem == "--demo":
        img = np.full((360, 480, 3), 230, np.uint8)
        cv2.circle(img, (120, 180), 60, (40, 40, 220), -1)          # circulo vermelho (BGR!)
        cv2.rectangle(img, (240, 100), (400, 260), (60, 180, 60), -1)  # retangulo verde
        cv2.putText(img, "demo", (300, 320), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 0), 2)
        return img
    if origem.startswith("http"):
        import requests
        dados = requests.get(origem, timeout=5).content                    # bytes do JPEG
        return cv2.imdecode(np.frombuffer(dados, np.uint8), cv2.IMREAD_COLOR)  # bytes -> matriz
    return cv2.imread(origem)

# Classificador Haar de rostos: no OpenCV 5 o XML nao vem mais junto. Baixe
# https://raw.githubusercontent.com/opencv/opencv/4.x/data/haarcascades/
haarcascade_frontalface_default.xml
# para a pasta python/modelos/. Sem ele, a deteccao de rostos e pulada.
import os
_xml = os.path.join(os.path.dirname(os.path.abspath(__file__)), "modelos",
"haarcascade_frontalface_default.xml")
cascata = cv2.CascadeClassifier(_xml) if os.path.exists(_xml) else None
if cascata is None: print("(sem haarcascade_frontalface_default.xml em modelos/: deteccao de rostos
desligada)")

def analisar(img):
    h, w = img.shape[:2]
    print(f"imagem {w}x{h}, {img.dtype}, pixel (0,0) BGR = {img[0,0]}")
    cinza = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)                    # 3 numeros por pixel -> 1
    bordas = cv2.Canny(cinza, 80, 160)                                 # onde o brilho muda rapido
    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)                         # matiz-saturacao-valor: cor
"separada" do brilho
    mascara = cv2.inRange(hsv, (0, 120, 70), (10, 255, 255)) | cv2.inRange(hsv, (170, 120, 70), (180, 255,
255))  # vermelho
    contornos, _ = cv2.findContours(mascara, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    saida = img.copy()
    for c in contornos:
        if cv2.contourArea(c) > 500:                                   # ignora manchinhas
            x, y, cw, ch = cv2.boundingRect(c)
            cv2.rectangle(saida, (x, y), (x + cw, y + ch), (0, 255, 255), 2)
            cv2.putText(saida, "vermelho", (x, y - 5), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 255), 2)
```

```
    rostos = cascata.detectMultiScale(cinza, 1.2, 5, minSize=(40, 40)) if cascata is not None else []
    for (x, y, cw, ch) in rostos: cv2.rectangle(saida, (x, y), (x + cw, y + ch), (255, 0, 0), 2)
    print(f"objetos vermelhos: {sum(cv2.contourArea(c) > 500 for c in contornos)}   rostos:
{len(rostos)}")
    return cinza, bordas, mascara, saida

if __name__ == "__main__":
    origem = sys.argv[1] if len(sys.argv) > 1 else "--demo"
    if origem == "--webcam":
        cap = cv2.VideoCapture(0)
        while True:
            ok, img = cap.read()
            if not ok: break
            cinza, bordas, mascara, saida = analisar(img)
            cv2.imshow("resultado", saida); cv2.imshow("bordas", bordas)
            if cv2.waitKey(1) & 0xFF == ord("q"): break
    else:
        img = carregar(origem)
        cinza, bordas, mascara, saida = analisar(img)
        for nome, im in [("original", img), ("cinza", cinza), ("bordas", bordas), ("mascara vermelho",
mascara), ("resultado", saida)]:
            cv2.imshow(nome, im)
        if "--salvar" in sys.argv: cv2.imwrite("resultado.jpg", saida); cv2.imwrite("bordas.jpg", bordas);
print("salvo resultado.jpg / bordas.jpg")
        else: cv2.waitKey(0)
    cv2.destroyAllWindows()
```

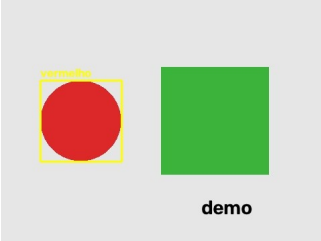


Figura 3 — Aula 4 — saída do `--demo`: a máscara HSV encontrou o objeto vermelho e desenhou o contorno.

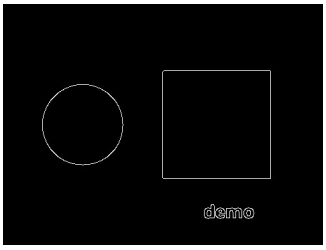


Figura 4 — Aula 4 — bordas (Canny): onde o brilho muda bruscamente.

Entenda o código

- **Cinza**: 0,299 R + 0,587 G + 0,114 B — o verde pesa mais porque o olho é mais sensível a ele (o mesmo motivo do filtro de Bayer).
- **Canny**: calcula o gradiente (quanto o brilho muda entre pixels vizinhos) e mantém só os máximos — é uma derivada em 2D.
- **HSV**: separa *matiz* (a cor), *saturação* (quão pura) e *valor* (brilho). Escolher “vermelho” em HSV funciona com sombra e luz; em RGB não. O vermelho está nas duas pontas do círculo (0° e 180°), por isso duas faixas.
- **Contornos**: regiões conectadas da máscara; `boundingRect` dá a caixa. `contourArea > 500` ignora ruído.

- **Haar cascade**: um classificador de 2001 treinado com milhares de rostos: procura padrões de claro/escuro (olhos mais escuros que as bochechas…) em janelas de vários tamanhos. É *aprendizado de máquina* clássico: rápido, mas frágil com rostos de lado.

Experimente

1. `python reconhecer_imagem.py http://192.168.4.1/capture` com um objeto vermelho na frente da câmera; mude as faixas HSV para detectar azul.
2. Rode `--webcam` e aproxime o rosto: o retângulo azul aparece? Vire de lado. Tampe um olho.
3. Mude os limiares do Canny (80, 160) para (20, 60) e (150, 300).
4. Imprima `img.shape` e `img[100:110, 100:110, 2]` — os valores de vermelho num quadradinho.

Desafios

1. **4.1 Semáforo de cor** — detecte vermelho, verde e azul; o objeto maior define a cor enviada ao ESP32 (`/led?cor=`) — o LED copia o objeto (prévia da Aula 5).
2. **4.2 Contador de objetos** — quantos objetos vermelhos há? Mostre o número na imagem; buzzer no PC (`winsound`) se passar de 3.
3. **4.3 Timelapse → vídeo** — leia as fotos do microSD (Aula 2) em ordem e gere `timelapse.avi` com `VideoWriter`.
4. **4.4 Detector de movimento no PC** — diferença entre quadros consecutivos (`cv2.absdiff`) + limiar + área: se mudou muito, salve a foto. Compare com o PIR: quando cada um é melhor?
5. **4.5 Rosto → LED** — rosto detectado = LED verde no ESP32; nenhum = apagado; mais de um = amarelo.

Aula 5 — Projeto: gestos com a mão acendem LEDs

O que você vai aprender

- Como um **modelo treinado** (rede neural) reconhece a mão: 21 pontos (landmarks) e um classificador de gestos.
- Arquitetura **câmera → rede → PC (inferência) → rede → atuador**.
- Estabilidade (votação de 4 em 5 quadros), latência e *watchdog* no ESP32.
- Mapear gestos em ações — e por que isso é a base de interfaces naturais.

Como o MediaPipe reconhece a mão

O modelo `gesture_recognizer.task` (Google) tem três etapas: (1) um **detector de palma** encontra a mão na imagem; (2) um **modelo de landmarks** estima 21 pontos 3D (pulso, articulações, pontas dos dedos); (3) um **classificador** olha a geometria desses pontos e diz o gesto: `Open_Palm`, `Closed_Fist`, `Pointing_Up`, `Victory`, `Thumb_Up`, `Thumb_Down`, `ILoveYou`. Cada etapa é uma rede neural treinada com milhões de imagens — nós só **usamos** o modelo. Isso é *inferência*; o *treinamento* aconteceu no Google. A saída vem com uma **confiança** (0–1).

Montagem

LED RGB (cátodo comum)	ESP32-CAM
R → 220 Ω	GPIO 12
G → 220 Ω	GPIO 13
B → 220 Ω	GPIO 14
comum	GND

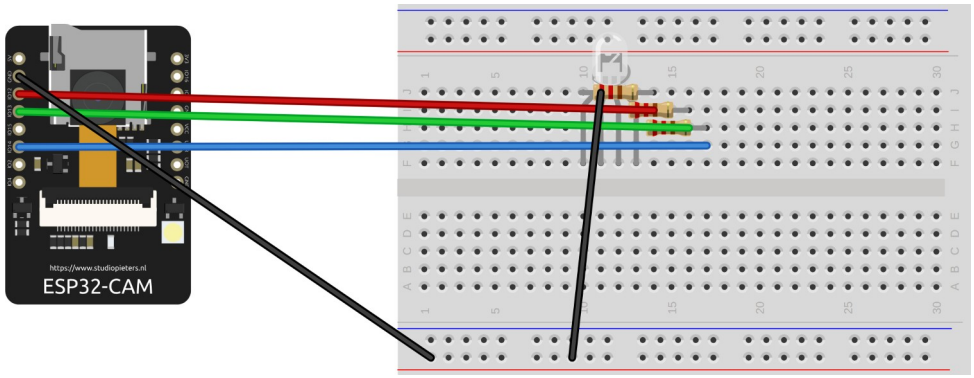


Figura 5 — Aula 5 — LED RGB nos GPIO 12/13/14 do ESP32-CAM (microSD não usado).

Código 4 — ESP32-CAM: olhos e LED

04_cam_gestos_leds.ino

```
/*
  04 - Projeto: gestos com a mao (reconhecidos em Python) acendem LEDs de cores diferentes
  Placa: AI Thinker ESP32-CAM      Arquivo extra: camera_aithinker.h


  Divisao de trabalho:
    ESP32-CAM  = olhos (video/foto pela rede) + maos (LED RGB)
    Python/PC  = cerebro (MediaPipe reconhece o gesto e manda a cor de volta por HTTP)
```

```
  Ligacoes (microSD nao usado, entao GPIO 12/13/14 estao livres):
    LED RGB catodo comum: R -> 220 -> GPIO 12   G -> 220 -> GPIO 13   B -> 220 -> GPIO 14   comum -> GND
  Rede: "Camera-ESP32" / http://192.168.4.1     (ou MODO_AP = false para entrar na rede da escola)
  Rotas: /capture (foto), :81/stream (video), /led?r=255&g=0&b=0, /led?cor=verde, /estado
*/
#include <WiFi.h>
#include "esp_http_server.h"
#include "camera_aithinker.h"

const bool MODO_AP = true;
const char* SSID_AP = "Camera-ESP32"; const char* SENHA_AP = "12345678";
const char* SSID_REDE = "NOME-DA-REDE"; const char* SENHA_REDE = "SENHA-DA-REDE";
const int PINO_R = 12, PINO_G = 13, PINO_B = 14;
httpd_handle_t servidorWeb = NULL, servidorStream = NULL;
int corR = 0, corG = 0, corB = 0;
String ultimoGesto = "nenhum";
unsigned long ultimoComando = 0;

void cor(int r, int g, int b) { corR = r; corG = g; corB = b; ledcWrite(PINO_R, r); ledcWrite(PINO_G, g);
ledcWrite(PINO_B, b); }

// cores por nome (o Python pode mandar o nome do gesto direto)
bool corPorNome(const String& n) {
  if (n == "vermelho" || n == "Closed_Fist")  cor(255, 0, 0);
  else if (n == "verde" || n == "Open_Palm")  cor(0, 255, 0);
  else if (n == "azul" || n == "Pointing_Up") cor(0, 0, 255);
  else if (n == "amarelo" || n == "Victory")  cor(255, 150, 0);
  else if (n == "roxo" || n == "Thumb_Up")    cor(160, 0, 255);
  else if (n == "ciano" || n == "ILoveYou")   cor(0, 255, 255);
  else if (n == "branco")                     cor(255, 255, 255);
  else if (n == "off" || n == "Thumb_Down" || n == "None") cor(0, 0, 0);
  else return false;
  return true;
}

static String argStr(httpd_req_t* req, const char* nome) {
  char url[128], v[32];
  if (httpd_req_get_url_query_str(req, url, sizeof(url)) == ESP_OK && httpd_query_key_value(url, nome, v,
sizeof(v)) == ESP_OK) return String(v);
  return "";
}
static esp_err_t captureHandler(httpd_req_t* req) {
  camera_fb_t* fb = esp_camera_fb_get(); if (!fb) { httpd_resp_send_500(req); return ESP_FAIL; }
  httpd_resp_set_type(req, "image/jpeg"); httpd_resp_set_hdr(req, "Access-Control-Allow-Origin", "*");
  esp_err_t r = httpd_resp_send(req, (const char*)fb->buf, fb->len); esp_camera_fb_return(fb); return r;
}
static esp_err_t streamHandler(httpd_req_t* req) {
  httpd_resp_set_type(req, "multipart/x-mixed-replace;boundary=frame"); char cab[64];
  while (true) {
    camera_fb_t* fb = esp_camera_fb_get(); if (!fb) break;
    size_t n = snprintf(cab, sizeof(cab), "\r\n--frame\r\nContent-Type: image/jpeg\r\nContent-Length:
%u\r\n\r\n", fb->len);
    esp_err_t r = httpd_resp_send_chunk(req, cab, n); if (r == ESP_OK) r = httpd_resp_send_chunk(req,
(const char*)fb->buf, fb->len);
    esp_camera_fb_return(fb); if (r != ESP_OK) break;
  }
  return ESP_OK;
}
static esp_err_t ledHandler(httpd_req_t* req) {
  String nome = argStr(req, "cor"), sr = argStr(req, "r");
  if (nome.length()) { corPorNome(nome); ultimoGesto = nome; }
  else if (sr.length()) cor(sr.toInt(), argStr(req, "g").toInt(), argStr(req, "b").toInt());
  ultimoComando = millis();
  Serial.printf("LED <- %s (%d,%d,%d)\n", nome.c_str(), corR, corG, corB);
  httpd_resp_set_hdr(req, "Access-Control-Allow-Origin", "*");
  return httpd_resp_send(req, "ok", HTTPD_RESP_USE_STRLEN);
}
static esp_err_t estadoHandler(httpd_req_t* req) {
  String j = "{\"gesto\":\"" + ultimoGesto + "\",\"r\":" + corR + ",\"g\":" + corG + ",\"b\":" + corB +
"}";
  httpd_resp_set_type(req, "application/json"); return httpd_resp_send(req, j.c_str(), j.length());
}
static esp_err_t paginaHandler(httpd_req_t* req) {
```

```
  static const char P[] = R"HTML(<!DOCTYPE html><html><head><meta charset="utf-8"><meta name="viewport"
content="width=device-width,initial-scale=1"><title>Gestos</title>
<style>body{font-family:sans-serif;background:#111;color:#eee;text-align:center;padding:10px}img{max-
width:100%;border-radius:8px}#c{width:80px;height:80px;border-radius:50%;margin:10px auto;border:3px solid
#444}</style></head>
<body><h2>ESP32-CAM - gestos</h2><img id="v"><div id="c"></div><p id="g">-</p>
<script>v.src=location.protocol+'//'+location.hostname+':81/stream';setInterval(async()=>{const d=await
(await fetch('/estado')).json();
c.style.background='rgb('+d.r+','+d.g+','+d.b+')';g.textContent='ultimo gesto:
'+d.gesto},500)</script></body></html>)HTML";
  httpd_resp_set_type(req, "text/html; charset=utf-8"); return httpd_resp_send(req, P,
HTTPD_RESP_USE_STRLEN);
}

void setup() {
  Serial.begin(115200);
  ledcAttach(PINO_R, 5000, 8); ledcAttach(PINO_G, 5000, 8); ledcAttach(PINO_B, 5000, 8);
  cor(0, 0, 40);
  if (!cameraIniciar(FRAMESIZE_VGA, 12)) { while (true) delay(1000); }     // VGA: bom para o MediaPipe e
rapido
  if (MODO_AP) { WiFi.softAP(SSID_AP, SENHA_AP); Serial.print("http://"); Serial.println(WiFi.softAPIP());
}
  else { WiFi.begin(SSID_REDE, SENHA_REDE); while (WiFi.status() != WL_CONNECTED) { delay(500);
Serial.print("."); } Serial.print("\nhttp://"); Serial.println(WiFi.localIP()); }
  httpd_config_t cfg = HTTPD_DEFAULT_CONFIG();
  httpd_uri_t us[] = { {"/", HTTP_GET, paginaHandler, NULL}, {"/capture", HTTP_GET, captureHandler, NULL},
{"/led", HTTP_GET, ledHandler, NULL}, {"/estado", HTTP_GET, estadoHandler, NULL} };
  if (httpd_start(&servidorWeb, &cfg) == ESP_OK) for (auto& u : us)
httpd_register_uri_handler(servidorWeb, &u);
  cfg.server_port = 81; cfg.ctrl_port = 32769;
  httpd_uri_t uStream = { "/stream", HTTP_GET, streamHandler, NULL };
  if (httpd_start(&servidorStream, &cfg) == ESP_OK) httpd_register_uri_handler(servidorStream, &uStream);
  cor(0, 0, 0);
}

void loop() {
  // se o Python parar de mandar comandos por 5 s, apaga (evita LED "preso")
  if (ultimoComando && millis() - ultimoComando > 5000) { cor(0, 0, 0); ultimoComando = 0; ultimoGesto =
"sem conexao"; }
  delay(100);
}
```

Script 3 — Python: o cérebro

gestos_leds.py

```
"""Projeto: gestos com a mao (MediaPipe) acendem cores no LED RGB do ESP32-CAM.

Uso:
    python gestos_leds.py                                  # camera: ESP32-CAM em 192.168.4.1
    python gestos_leds.py --esp 192.168.1.50               # outro IP
    python gestos_leds.py --webcam                         # testa com a webcam do PC (sem ESP32)
    python gestos_leds.py --webcam --sem-esp               # so reconhece, nao manda nada
Tecla q sai.

Gestos que o modelo do Google reconhece -> cor enviada ao ESP32 (rota /led?cor=...):
    Open_Palm (mao aberta) -> verde      Closed_Fist (punho) -> vermelho   Pointing_Up (dedo p/ cima) ->
azul
    Victory (V) -> amarelo               Thumb_Up (joinha) -> roxo         ILoveYou -> ciano
    Thumb_Down (joinha p/ baixo) -> apaga
Bibliotecas: opencv-python, mediapipe, requests.  Modelo: modelos/gesture_recognizer.task
"""
import sys, os, time, threading, cv2, numpy as np, requests
import mediapipe as mp
from mediapipe.tasks.python import vision, BaseOptions

ESP = "192.168.4.1"
if "--esp" in sys.argv: ESP = sys.argv[sys.argv.index("--esp") + 1]
USAR_WEBCAM = "--webcam" in sys.argv
ENVIAR = "--sem-esp" not in sys.argv
```

```
MODELO = os.path.join(os.path.dirname(os.path.abspath(__file__)), "modelos", "gesture_recognizer.task")
CORES = {"Open_Palm": "verde", "Closed_Fist": "vermelho", "Pointing_Up": "azul", "Victory": "amarelo",
         "Thumb_Up": "roxo", "ILoveYou": "ciano", "Thumb_Down": "off"}
CORES_BGR = {"verde": (0, 255, 0), "vermelho": (0, 0, 255), "azul": (255, 0, 0), "amarelo": (0, 200, 255),
"roxo": (255, 0, 160), "ciano": (255, 255, 0), "off": (80, 80, 80)}

# ---------- fonte de video ----------
class FonteESP:
    """le fotos /capture em sequencia (mais robusto que o stream quando o Wi-Fi oscila)"""
    def __init__(self, ip): self.url = f"http://{ip}/capture"
    def ler(self):
        try:
            dados = requests.get(self.url, timeout=3).content
            return cv2.imdecode(np.frombuffer(dados, np.uint8), cv2.IMREAD_COLOR)
        except requests.RequestException: return None
class FonteWebcam:
    def __init__(self): self.cap = cv2.VideoCapture(0)
    def ler(self):
        ok, q = self.cap.read(); return q if ok else None

# ---------- envio ao ESP32 em segundo plano (nao trava o video) ----------
ultimaCorEnviada = None
def enviar(cor):
    global ultimaCorEnviada
    if not ENVIAR or cor == ultimaCorEnviada: return
    ultimaCorEnviada = cor
    def _go():
        try: requests.get(f"http://{ESP}/led", params={"cor": cor}, timeout=2)
        except requests.RequestException: print("(ESP32 nao respondeu)")
    threading.Thread(target=_go, daemon=True).start()

# ---------- reconhecedor ----------
opcoes = vision.GestureRecognizerOptions(base_options=BaseOptions(model_asset_path=MODELO), num_hands=1,
                                         min_hand_detection_confidence=0.5, min_tracking_confidence=0.5)
reconhecedor = vision.GestureRecognizer.create_from_options(opcoes)

def main():
    fonte = FonteWebcam() if USAR_WEBCAM else FonteESP(ESP)
    print("fonte:", "webcam" if USAR_WEBCAM else f"ESP32-CAM {ESP}", "| envio:", "sim" if ENVIAR else
"nao")
    historico = []                                      # os ultimos N gestos: so aceita se for estavel
    while True:
        quadro = fonte.ler()
        if quadro is None: time.sleep(0.3); continue
        rgb = cv2.cvtColor(quadro, cv2.COLOR_BGR2RGB)     # MediaPipe usa RGB; OpenCV usa BGR
        res = reconhecedor.recognize(mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb))
        gesto, conf = "None", 0.0
        if res.gestures:
            g = res.gestures[0][0]; gesto, conf = g.category_name, g.score
            for lm in res.hand_landmarks[0]:              # desenha os 21 pontos da mao
                cv2.circle(quadro, (int(lm.x * quadro.shape[1]), int(lm.y * quadro.shape[0])), 4, (0, 255,
255), -1)
        historico.append(gesto); historico = historico[-5:]
        estavel = historico.count(gesto) >= 4             # 4 de 5 quadros iguais = gesto firme
        cor = CORES.get(gesto, None)
        if estavel and cor: enviar(cor)
        cv2.rectangle(quadro, (0, 0), (quadro.shape[1], 40), CORES_BGR.get(cor, (40, 40, 40)), -1)
        cv2.putText(quadro, f"{gesto} ({conf:.2f}) -> {cor or '-'}", (10, 28), cv2.FONT_HERSHEY_SIMPLEX,
0.8, (255, 255, 255), 2)
        cv2.imshow("gestos -> LEDs (q sai)", quadro)
        if cv2.waitKey(1) & 0xFF == ord("q"): break
    cv2.destroyAllWindows()

if __name__ == "__main__":
    main()
```

Entenda o sistema

- O Python pega uma foto por vez em `/capture` (mais robusto que o stream quando o Wi-Fi oscila) — ~5–8 quadros/s em VGA, suficiente para gestos.
- `GestureRecognizer.recognize()` devolve gestos e landmarks; desenhamos os 21 pontos para **ver o que o modelo vê**.
- **Votação**: só aceita um gesto se ele aparecer em 4 dos últimos 5 quadros — elimina tremidas.
- O envio ao ESP32 roda numa **thread** e só quando a cor muda: o vídeo não trava esperando a rede.
- No ESP32, `ledHandler` traduz gesto → cor e o `loop()` apaga o LED se o PC ficar 5 s sem falar (**watchdog**): sistema seguro por padrão.
- Teste sem hardware: `python gestos_leds.py --webcam --sem-esp`.

Experimente

1. Faça cada um dos 7 gestos e anote a confiança. Qual é o mais "fácil" para o modelo? Com luz de costas?
2. Afaste-se 3 m: até onde funciona em VGA? E em SVGA?
3. Tire a votação (`>= 1`): o LED "pisca" entre cores? Por quê?
4. Meça a latência: do gesto ao LED, com um cronômetro no celular filmando os dois.

Desafios

1. **5.1 Duas mãos** — `num_hands=2`: mão esquerda controla a cor, direita o brilho (punho = apaga).
2. **5.2 Senha por gestos** — a sequência `Victory, Open_Palm, Thumb_Up` em 5 s "destrava" (LED verde + bipe); qualquer outra, vermelho. Máquina de estados!
3. **5.3 Contagem de dedos** — use `hand_landmarker.task` e conte dedos levantados pela geometria (ponta acima da articulação) → número de piscadas no LED.
4. **5.4 Gesto → servo** — junte com a aula de servos: `Pointing_Up` move o braço para a pose 1, `Victory` para a pose 2 (comando por HTTP ao ESP32 do braço).
5. **5.5 Tudo no ESP32?** — pesquise "ESP32-CAM face detection" (a Espressif tem detecção de rostos rodando no próprio chip). O que dá para fazer sem o PC, e o que não dá? Discuta limites de memória e velocidade.

Para ir além

- **ESP32-S3-CAM / ESP32-S3-EYE**: mais RAM e aceleração de IA — roda modelos pequenos (TensorFlow Lite Micro) na própria placa.
- **Treinar o seu gesto**: o MediaPipe Model Maker permite adicionar gestos próprios com ~100 fotos.
- **YOLO** (Ultralytics) no PC: detecta 80 tipos de objetos em tempo real a partir do stream do ESP32-CAM.
- **Home Assistant / Frigate**: integrar a câmera numa central de automação com gravação por detecção de objetos.

Glossário

Termo	Significado
CMOS / pixel / exposição	Sensor de imagem; célula sensível à luz; tempo de coleta de luz.
Filtro de Bayer	Mosaico RGB sobre os pixels; a cor é interpolada.
JPEG	Compressão com perdas; qualidade 0–63 no ESP32 (menor = melhor).
MJPEG	Vídeo = sequência de JPEGs num único fluxo HTTP.
PSRAM	RAM externa do ESP32-CAM; guarda quadros grandes.
Frame buffer	Área de memória com um quadro; pegar e **devolver**.
PIR / Fresnel	Sensor infravermelho passivo; lente que fatia o campo de visão.
SD_MMC 1-bit	Modo do cartão que usa menos pinos.
NAT / IP privado	Roteador que esconde a rede local da internet.
Túnel	Conexão que atravessa o NAT (Tailscale, ngrok).
Apps Script	JavaScript rodando nos servidores do Google, com acesso ao Drive.
Base64 / multipart	Formas de mandar bytes dentro de texto / de formulário HTTP.
BGR / HSV	Ordem de cores do OpenCV / espaço matiz-saturação-valor.
Canny / contorno	Detector de bordas / região conectada.
Haar cascade	Classificador clássico de padrões claro-escuro.
Landmark	Ponto-chave detectado (21 por mão).
Inferência × treinamento	Usar um modelo pronto × criar o modelo com dados.
Latência	Tempo entre o gesto e a reação.
Watchdog	Rotina que desliga se o controle sumir.

Anexo — Checklist

- [ ] Placa **AI Thinker ESP32-CAM** selecionada; `camera_aithinker.h` na pasta
- [ ] 5 V com corrente (≥ 500 mA); cabo flat travado; PSRAM detectada no Serial
- [ ] IO0 no GND só para gravar; RST depois
- [ ] microSD FAT32; PIR estabilizado (60 s)
- [ ] Wi-Fi com internet + URL do Apps Script + token/chat_id (Aula 3)
- [ ] Python com opencv, mediapipe, requests; modelos em `python/modelos/`
- [ ] Celular/PC na rede `Camera-ESP32` (ou mesmo Wi-Fi em modo estação)